

吴若琪·技术统筹 + 创新点开发·复赛专业计划

你的核心价值

你是团队的“技术天花板”。你的任务不是写业务代码，而是：1. 找到让 Cat Life 真正用上大模型的路径 2. 探索 AI 行为识别从规则引擎到智能模型的技术路线 3. 确保提交材料中有“核心功能调用大模型的代码包”

提交目标

大模型调用代码包：证明 Cat Life 不只是规则引擎，而是真正调用了 AI 能力。

Week 1 (5/26-6/1)：大模型调研 + 方案设计

调研输出（给团队看的决策文档）

模型选型对比：

模型	API	成本	适用场景	推荐
DeepSeek-V3	开放 API	极低	猫咪对话、状态分析	
豆包 (Doubao)	火山引擎	中等	中文场景友好	
Qwen2.5	阿里云/本地	免费	本地推理	
GPT-4o-mini	OpenAI	低	多模态识别	
GLM-4-Flash	智谱	免费	轻量调用	

大模型在 Cat Life 中的 3 个可落地场景：

场景A（优先级P0）：智能专注判定

输入：用户近30秒的行为特征（点击率/滑动率/停顿时间）

输出：专注倾向评分 + 建议状态（普通/过渡/专注）

调用方式：云端API，每30秒1次（成本可控）

示例Prompt：

"用户在最近30秒内点击2次，滑动0次，停顿28秒。

根据Cat Life专注判定规则，输出JSON: {state: 'focus', score: 0.87, reason: '...'}"

场景B（优先级P1）：猫咪对话

输入：用户输入文本

输出：符合猫咪性格的简短回复（暖萌风格，<50字）

调用方式：按需调用

场景C（优先级P2）：专注总结

输入：一段专注时长的行为数据

输出：人性化的专注小结

调用方式：专注结束时调用

推荐方案

- 主力模型：DeepSeek-V3（成本低、中文好、API 稳定）
- 备选：豆包 lite（火山引擎，学生免费额度）
- 调用方式：HTTP POST → JSON Response
- 客户端：Unity C# → UnityWebRequest → API

技术架构图

[Android App]

↓ 行为数据采集

```

[Unity Layer]
  ↓ UnityWebRequest
[API Gateway]
  ↓
[DeepSeek / 豆包]
  ↓ JSON Response
[Unity Layer]
  ↓ 驱动状态机
[猫咪行为+UI反馈]

```

Week 2 (6/2-6/8) : 调用链路打通

核心代码 (给陈泓森集成)

```

// LLMClient.cs - 大模型调用封装
using UnityEngine;
using UnityEngine.Networking;
using System;
using System.Collections;

[System.Serializable]
public class LLMRequest {
    public string model = "deepseek-chat";
    public Message[] messages;
    public bool stream = false;
    public float temperature = 0.3;
    public int max_tokens = 256;
}

[System.Serializable]
public class Message {
    public string role;
    public string content;
}

[System.Serializable]
public class LLMResponse {
    public Choice[] choices;
}

[System.Serializable]
public class Choice {
    public Message message;
}

public class LLMClient : MonoBehaviour {
    private const string API_URL = "https://api.deepseek.com/v1/chat/completions";
    private const string API_KEY = "YOUR_DEEPSEEK_API_KEY";

    public void AnalyzeFocusState(BehaviorData data, Action<string> callback) {
        StartCoroutine(CallLLM(BuildPrompt(data), callback));
    }

    string BuildPrompt(BehaviorData d) {
        return $"你 是 Cat Life 的专注状态分析器。根据以下行为数据判断用户状态。
规则：点击率>10/min=活跃，停顿>20s=趋向专注，连续低频=专注。

```

数据：点击{d.clickCount}次，滑动{d.swipeCount}次，停顿{d.idleSeconds}秒。

输出 JSON 格式：{"state":"normal/transition/focus","score":0.0-1.0}";
}

```
IEnumerator CallLLM(string prompt, Action<string> callback) {  
    var req = new LLMRequest {  
        messages = new[] { new Message { role = "user", content = prompt } }  
    };  
  
    using var web = new UnityWebRequest(API_URL, "POST");  
    var json = JsonUtility.ToJson(req);  
    var body = new System.Text.UTF8Encoding().GetBytes(json);  
    web.uploadHandler = new UploadHandlerRaw(body);  
    web.downloadHandler = new DownloadHandlerBuffer();  
    web.SetRequestHeader("Content-Type", "application/json");  
    web.SetRequestHeader("Authorization", $"Bearer {API_KEY}");  
  
    yield return web.SendWebRequest();  
  
    if (web.result == UnityWebRequest.Result.Success) {  
        var resp = JsonUtility.FromJson<LLMResponse>(web.downloadHandler.text);  
        callback?.Invoke(resp.choices[0].message.content);  
    }  
}  
  
public struct BehaviorData {  
    public int clickCount, swipeCount;  
    public float idleSeconds;  
}
```

Week 3 (6/9-6/15)：创新点落地 + 联调

创新点文档（用于 PPT/答辩）

1. LLM-powered 状态识别：不靠死阈值，用大模型语义理解行为模式
2. 猫咪对话：大模型驱动的个性化猫咪回复
3. 专注洞察：大模型分析专注模式，生成个性化建议

代码包整理

```
llm_code_package/  
    README.md          # 使用说明  
    LLMClient.cs       # Unity C# 调用封装  
    PromptTemplates.cs # Prompt模板管理  
    BehaviorAnalyzer.cs # 行为数据预处理  
    config.json        # API配置(不含key)  
    requirements.txt    # 依赖说明
```

Week 4 (6/16-6/23)：最终交付

- 代码包整理 + 注释
- 撰写技术说明文档
- 协助傅钧瀚准备技术部分的答辩内容
- 最终联调把关

全局统筹职责

每周做

1. 周日检查各成员进度（按检查点）
2. 识别阻塞点，调配资源
3. 与傅钧烨对齐资产 →Unity 的交接
4. 与陈泓森对齐 Unity→Android 的交接

技术决策权

- 模型选型有最终决定权
- API Key 管理由你负责
- 代码质量标准由你制定